

Tecnológico de Monterrey

Instituto Tecnológico y de Estudios Superiores de Monterrey

Construye | Tests + Chequeos de seguridad básicos integrados al repositorio

Matrícula: A01795954
Nombre: Manuel Alejandro Cardona Esquivel
Profesor: Iván Reyes Amezcua

1 de noviembre del 2026

Introducción

El proyecto integrador trabaja con los datos abiertos de MiBici, el sistema de bicicleta pública de la zona metropolitana de Guadalajara, para responder tres preguntas: cuántos viajes se iniciarán en cada estación al día siguiente (P1), qué estaciones corren el riesgo de quedarse sin bicicletas o sin anclajes en las próximas horas (P2) y qué grupos de estaciones comparten patrones de uso (P3). Hasta la entrega anterior, el repositorio ya contaba con una ingesta reproducible, el seguimiento de experimentos en MLflow y un registro de modelos, pero no tenía forma automática de confirmar que el código, los datos y las dependencias siguieran en buen estado después de cada cambio.

En esta actividad se agregó esa capa de verificación. Se construyó una suite de pruebas unitarias con PyTest que incluye la comprobación de que el entrenamiento es reproducible, se integró una validación de calidad e integridad de los datos con DeepChecks y se revisó la seguridad del repositorio con herramientas de análisis de código y de dependencias. Todo quedó organizado para ejecutarse con pocos comandos y documentado en el README, junto con la evidencia de cada ejecución.

Pruebas unitarias con PyTest

Las pruebas viven en la carpeta *tests/* y se ejecutan con un solo comando, *python -m pytest*, desde la raíz del repositorio. La configuración de PyTest quedó en *pyproject.toml*, así que no hace falta indicar rutas ni opciones adicionales. Una decisión importante fue que ninguna prueba dependiera de los datos reales ni de un servidor de MLflow: cada prueba genera viajes sintéticos con la misma semilla que usa el proyecto, lo que permite correr la suite completa en unos diez segundos y obtener siempre el mismo resultado, sin importar la computadora.

La suite cubre las funciones de las que dependen los tres modelos: la ingesta de datos, la construcción de variables de cada pregunta y las utilidades de entrenamiento y evaluación. En total se ejecutan 149 pruebas que pasan sin errores. La Tabla 1 resume qué revisa cada archivo.

Tabla 1

Archivos de prueba y funciones que verifican

Archivo	Pruebas	Qué se verifica
test_dataset.py	19	Normalización de nombres de columna con acentos y espacios, fechas inválidas, eliminación de datos personales, lectura de archivos en UTF-8 y latin-1, cálculo del hash de versión y descarga idempotente con la red simulada.

Archivo	Pruebas	Qué se verifica
test_features_p1.py	12	Panel diario de viajes por estación: días sin viajes, límites del rango de fechas, calendario y días feriados, rezagos y medias móviles que solo usan días anteriores.
test_features_p2.py	14	Panel horario y etiqueta de riesgo: umbral exacto, estaciones con capacidad cero, horizontes inválidos e histórico calculado sin información futura.
test_features_p3.py	13	Perfil de uso de cada estación: filtros de duración, criterios de elegibilidad, proporciones por bloque horario e índices de entropía.
test_modeling_utils.py	51	Lectura de hiperparámetros, etiquetas de MLflow, particiones temporales, métricas, elección del umbral de decisión y construcción de los modelos de las tres preguntas.
test_reproducibilidad.py	15	Reproducibilidad del entrenamiento de P1, P2 y P3 con semilla fija.
test_seguridad.py	25	Archivos sensibles fuera del repositorio y ausencia de credenciales (se describe en la sección de seguridad).

Además de los casos normales, cada archivo incluye casos límite que en la práctica son los que suelen romper un pipeline: tablas vacías, fechas que no se pueden interpretar, estaciones sin historial suficiente, capacidades en cero, valores que caen justo en el umbral y parámetros fuera de rango. También hay pruebas que verifican que no haya fuga de información. Por ejemplo, en P2 se altera a propósito el flujo de bicicletas posterior a la hora de predicción y se confirma que las variables calculadas para esa hora no cambian, lo que demuestra que el modelo nunca ve datos del futuro.

Reproducibilidad del entrenamiento

La prueba de reproducibilidad sigue el mismo camino que los pipelines reales: genera los datos, construye las variables con las funciones del proyecto, crea el modelo con la misma fábrica que se usa en producción, lo entrena y lo evalúa. Este recorrido se hace dos veces desde cero y se exige que las predicciones y las métricas coincidan exactamente, no solo que sean parecidas. Se comprobaron todos los modelos del proyecto: el baseline, la regresión de Poisson, el gradient boosting y el bosque aleatorio de P1; la regresión logística, el gradient boosting y el bosque aleatorio de P2, incluido el umbral de decisión elegido en validación; y K-Means, la mezcla gaussiana y el clustering aglomerativo de P3.

Para que la prueba tenga sentido, también se incluyó un control negativo: al cambiar la semilla, el bosque aleatorio sí produce resultados distintos. Si el resultado fuera igual con cualquier semilla, la prueba no estaría demostrando nada. Por último, se verifica que al clonar

un modelo, como ocurre en las búsquedas de hiperparámetros, la semilla se conserva. Las Figuras 1 y 2 muestran la ejecución de la suite completa y de las pruebas de reproducibilidad.

Figura 1

Ejecución de la suite completa de pruebas en el entorno principal

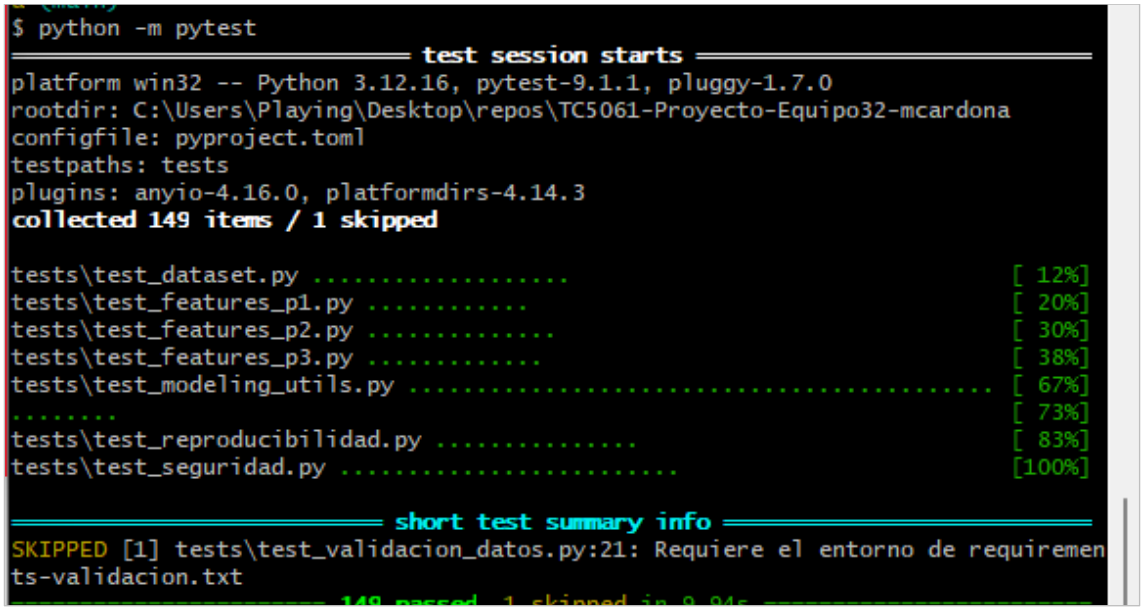
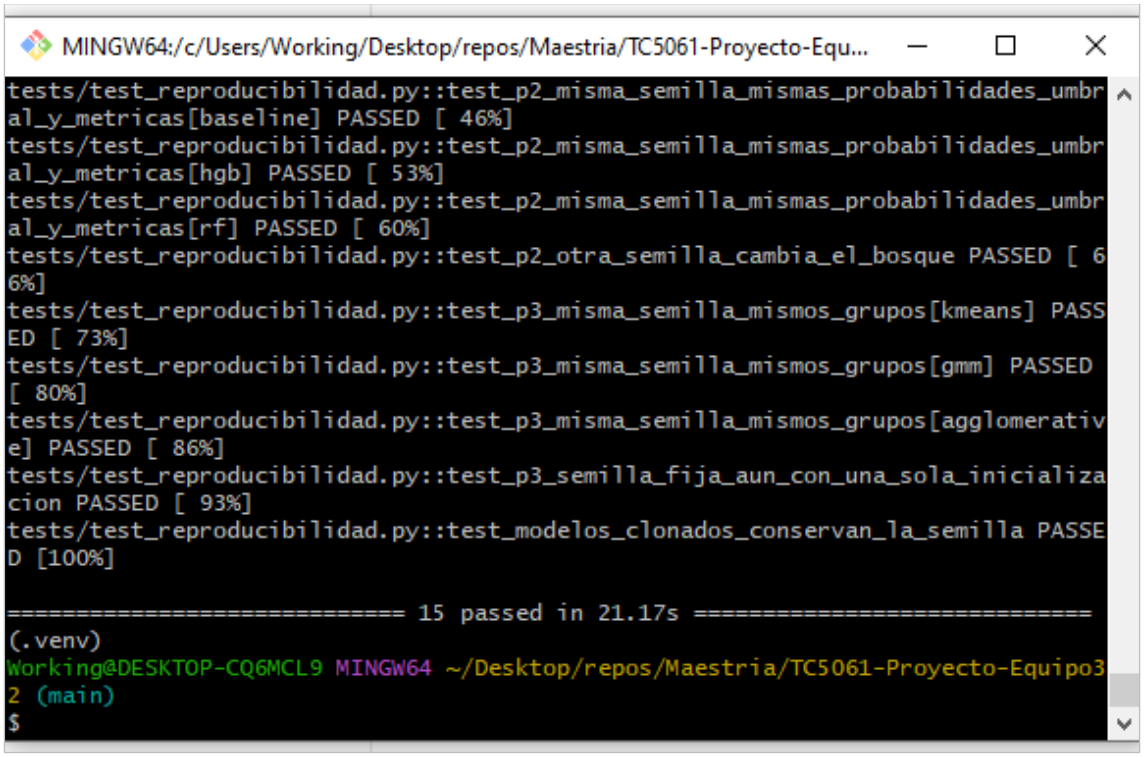


Figura 2

Pruebas de reproducibilidad del entrenamiento para P1, P2 y P3



Validación de datos con DeepChecks

La validación de datos se concentró en un módulo nuevo, *mibici/validacion_datos.py*, que revisa exactamente los mismos datos que consumen los modelos. Para lograrlo, construye los conjuntos de entrenamiento y de prueba de cada pregunta con las mismas funciones de variables que usan los pipelines, en lugar de revisar una copia aparte que podría no coincidir con lo que realmente se entrena. Se validan cinco conjuntos: la tabla de viajes, con 4,532,032 registros de 2025; el catálogo de 484 estaciones; y los conjuntos de enero a septiembre contra noviembre y diciembre de P1, P2 y P3.

A cada conjunto se le aplica la suite de integridad de DeepChecks, que busca valores faltantes, nulos escritos de distintas formas, columnas con tipos mezclados, filas duplicadas, etiquetas contradictorias y valores atípicos. Entre los periodos de entrenamiento y prueba se aplica además la suite de comparación, que mide el drift de cada variable y de la etiqueta, detecta categorías nuevas y revisa si hay fuga temporal entre ambos periodos. Como DeepChecks no conoce las reglas del negocio, el módulo también calcula chequeos exactos sobre la tabla completa: viajes con duración negativa o fuera de rango, viajes duplicados con distinto identificador, estaciones que no existen en el catálogo y estaciones con capacidad cero.

Durante la integración apareció un problema de compatibilidad. La versión más reciente de DeepChecks no funciona con las versiones de numpy, pandas y scikit-learn que usa el proyecto para entrenar. En lugar de parchear la librería o bajar de versión todo el entorno de entrenamiento, se creó un entorno aislado con Python 3.12 y versiones fijas, descrito en *requirements-validacion.txt*, que solo se usa para la validación. Así el entrenamiento conserva sus dependencias y la validación corre con una configuración estable.

No todos los hallazgos tienen el mismo peso. Los chequeos que indican un error real en los datos o una fuga entre entrenamiento y prueba se marcaron como críticos: valores faltantes, nulos o tipos mezclados, etiquetas en conflicto y traslape de fechas. Si alguno falla, el proceso termina con error. El resto se documenta en un resumen, pero no detiene la ejecución. Con los datos reales no hubo ninguna falla crítica. La Tabla 2 recoge los hallazgos más relevantes y cómo se interpretaron.

Tabla 2

Hallazgos de la validación con los datos reales

Hallazgo	Valor	Interpretación y tratamiento
Viajes duplicados con distinto identificador	1,076	Representan menos del 0.1 % de los viajes. Se documentan y se conservan, porque su efecto en los conteos es despreciable.
Viajes de menos de 60 segundos	117,226	Corresponden a desanclajes fallidos; los pipelines ya los descartan antes de construir variables.
Viajes de más de 24 horas	6	Bicicletas que no se anclaron correctamente; también se descartan.
Estaciones con capacidad cero	4	P2 las excluye porque sin capacidad no se puede definir el umbral de riesgo.
Nulos, identificadores repetidos, duraciones negativas y estaciones fuera del catálogo	0	Sin hallazgos.
Drift en la antigüedad de la estación (P1)	KS = 0.99	Es esperado: la variable cuenta los días desde la primera actividad de la estación y siempre es mayor en los meses de prueba.
Variables con correlación mayor a 0.9 (P1, P2 y P3)	Varios pares	Es esperado por la forma en que se construyen los rezagos y las medias; los modelos de árboles lo toleran y el modelo lineal usa regularización.
Muestras de prueba repetidas en entrenamiento (P2)	11.8 %	No es fuga: son horas distintas que tienen el mismo vector de conteos, sobre todo horas sin movimiento. Los chequeos de fechas confirman que los periodos no se traslapan.

Para confirmar que la validación detecta problemas cuando existen, se escribió un archivo de pruebas propio, *tests/test_validacion_datos.py*. En él se inyectan a propósito valores faltantes, nulos mezclados, tipos mezclados, filas duplicadas, drift y fechas traslapadas, y se verifica que cada chequeo los marque; con datos limpios, los mismos chequeos deben pasar. El archivo también ejecuta la validación completa de principio a fin y revisa los chequeos críticos con los datos reales. Las 11 pruebas pasan, como se ve en la Figura 4.

Figura 3

Ejecución de la validación de datos con cero fallas críticas

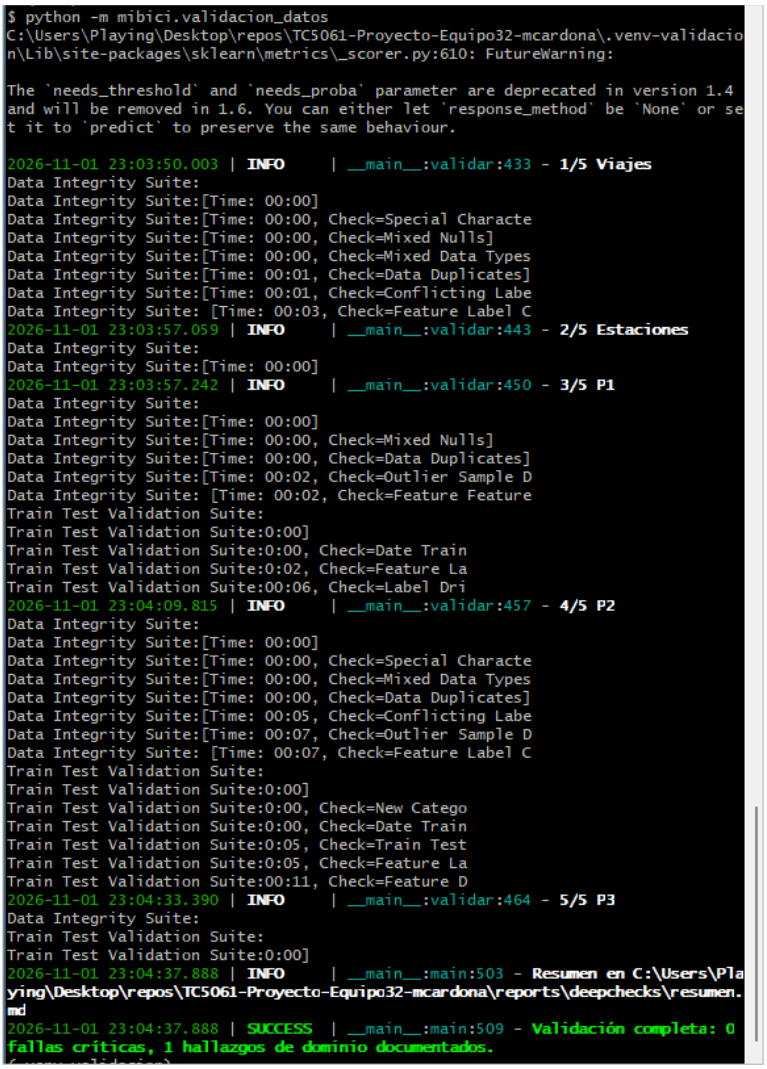
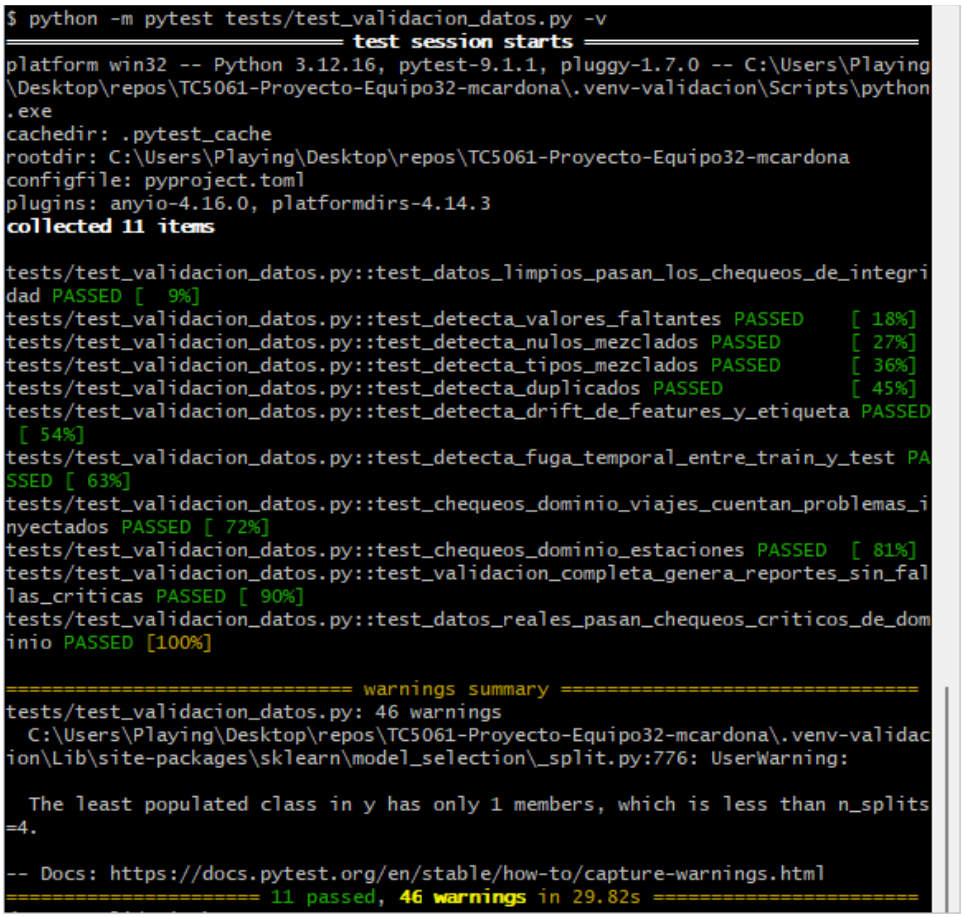


Figura 4

Pruebas de la validación de datos en el entorno aislado



Chequeos de seguridad

La revisión de seguridad se dividió en tres partes: el código del proyecto, las dependencias y el manejo de credenciales y datos sensibles. Las salidas de cada herramienta se guardaron en *reports/security/*, junto con un documento que explica cada hallazgo y la decisión que se tomó.

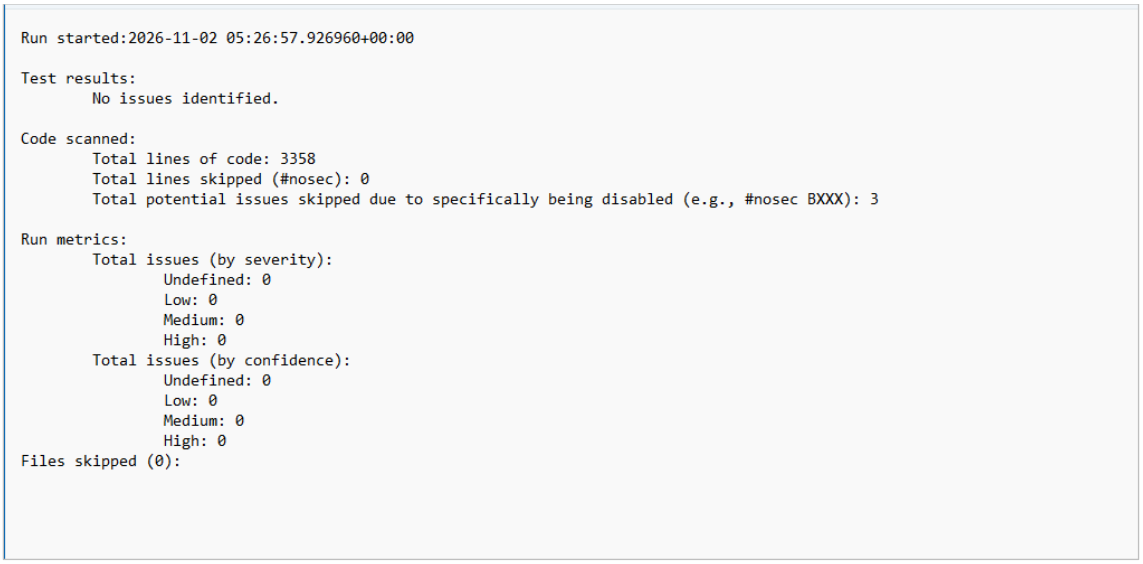
Análisis estático del código

Se ejecutó bandit sobre el paquete *mibici*. La primera corrida no encontró problemas de severidad media ni alta, pero sí trece avisos de severidad baja. Todos venían del mismo lugar: una función, repetida en los pipelines de P1, P2 y P3, que ejecuta Git para registrar en MLflow el commit con el que se entrenó cada modelo. Bandit marcaba el uso de subprocesos y el hecho de llamar a Git sin su ruta completa, lo que en teoría permitiría ejecutar un programa falso con el mismo nombre si se colocara antes en la ruta del sistema.

La función se reescribió en un solo lugar y ahora obtiene la ruta absoluta del ejecutable antes de llamarlo. El uso de subprocesos se mantuvo porque es necesario, pero se dejó justificado en el propio código: los argumentos son fijos, no se usa una consola intermedia y ningún dato externo llega al comando. Con este cambio, el análisis terminó sin ningún problema identificado (Figura 5).

Figura 5

Resultado del análisis estático con bandit



```
Run started:2026-11-02 05:26:57.926960+00:00

Test results:
        No issues identified.

Code scanned:
        Total lines of code: 3358
        Total lines skipped (#nosec): 0
        Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 3

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
Files skipped (0):
```

Auditoría de dependencias

Las dependencias se revisaron con pip-audit, que compara cada paquete y su versión contra las bases públicas de vulnerabilidades conocidas. Se auditaron por separado el entorno de entrenamiento y el de validación. La Tabla 3 resume lo que se encontró y lo que se hizo en cada caso.

Tabla 3

Resultados de la auditoría de dependencias

Entorno	Hallazgo	Acción
Entrenamiento	MLflow 3.17.1 con una vulnerabilidad publicada (CVE-2026-53995)	Se actualizó a la versión 3.17.2 y se volvió a correr la suite completa de pruebas. La auditoría final no reporta vulnerabilidades.
Validación	pyarrow 21.0.0 con una vulnerabilidad publicada	Se actualizó a la versión 23.0.1, que sigue siendo compatible con el resto del entorno.
Validación	setuptools 80.10.2 con tres avisos	Riesgo aceptado: DeepChecks requiere una versión anterior a la 81 y los avisos solo afectan la construcción de paquetes, algo que el proyecto no hace en ese entorno.

El caso de setuptools se dejó documentado como riesgo aceptado y no como algo pasado por alto. Las vulnerabilidades reportadas aparecen al generar distribuciones de código fuente o al instalar paquetes con un mecanismo antiguo, y el entorno de validación solo se usa de forma local para revisar datos, sin desplegarse ni exponerse como servicio. Este punto se revisará cuando DeepChecks publique una versión compatible con setuptools más reciente.

Figura 6

Auditoría del entorno de entrenamiento sin vulnerabilidades conocidas

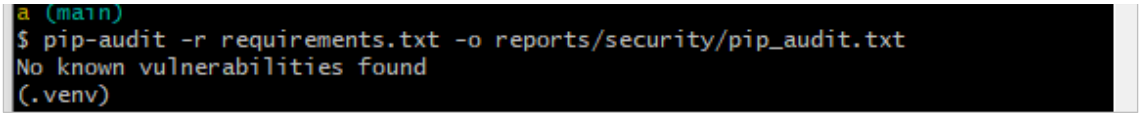
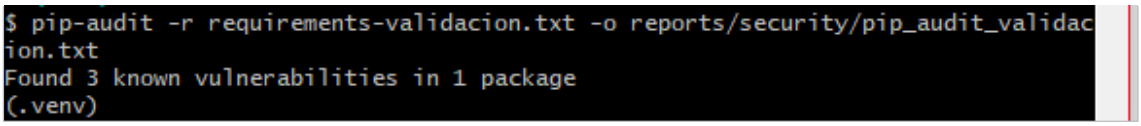


Figura 7

Auditoría del entorno de validación con el riesgo aceptado de setuptools



Credenciales y datos sensibles

Para que ninguna credencial termine en el repositorio, la configuración que cambia de una máquina a otra, como la dirección del servidor de MLflow o un usuario y contraseña, se lee de variables de entorno o de un archivo *.env* local. Ese archivo está excluido de Git, y en su lugar se versiona una plantilla, *.env.example*, con las variables vacías. El *.gitignore* se amplió para cubrir también llaves y certificados, la base de datos y los artefactos de MLflow, los datos descargados y los entornos virtuales.

Estas reglas no se quedaron solo en la configuración: se agregó un archivo de pruebas, *tests/test_seguridad.py*, que las comprueba en cada ejecución. Las pruebas confirman que no hay archivos sensibles versionados, que el *.gitignore* cubre cada tipo de archivo de riesgo y que ni los archivos actuales ni el historial completo de Git, en todas sus ramas, contienen credenciales con formatos conocidos, como llaves de acceso a la nube, llaves privadas, tokens o contraseñas escritas en el código. Para evitar falsos negativos, primero se valida que el detector sí encuentre secretos de ejemplo y que no marque código seguro. Respecto a los datos personales, la ingesta elimina desde el inicio el identificador del usuario, el género y el año de nacimiento, por lo que esos datos no llegan a los archivos procesados ni a MLflow.

Organización y ejecución

Toda la capa de calidad se puede ejecutar con pocos comandos, que quedaron documentados en una sección nueva del README con instrucciones paso a paso y las capturas de cada ejecución. Los reportes generados se guardan en *reports/deepchecks/* y *reports/security/*; los HTML de DeepChecks se excluyen de Git por su tamaño y se pueden regenerar en cualquier momento. La Tabla 4 resume los comandos.

Tabla 4

Comandos para ejecutar las pruebas y los chequeos

Objetivo	Comando	Entorno
Suite completa de pruebas	python -m pytest	Entrenamiento
Validación de datos	python -m mibici.validacion_datos	Validación
Pruebas de la validación	python -m pytest tests/test_validacion_datos.py	Validación
Análisis del código	bandit -r mibici	Entrenamiento
Auditoría de dependencias	pip-audit -r requirements.txt	Entrenamiento
Pruebas y seguridad en conjunto	make check	Entrenamiento

El trabajo se integró por etapas. Las pruebas, la validación de datos, los chequeos de seguridad y la documentación se desarrollaron cada uno en su propia rama, con commits descriptivos, y se incorporaron a la rama principal mediante pull requests. Así el historial de Git muestra con claridad qué se agregó en cada paso.

Conclusiones

Con esta entrega, el repositorio pasó de depender de revisiones manuales a tener una verificación automática que cubre el código, los datos y la seguridad. Las pruebas unitarias protegen las funciones de las que dependen los modelos y confirman que el entrenamiento es reproducible. La validación con DeepChecks permite saber, antes de entrenar, si los datos llegaron completos y consistentes, y deja por escrito por qué algunos hallazgos, como el drift en la antigüedad de las estaciones, son propios del problema y no errores. La revisión de seguridad corrigió una vulnerabilidad real en una dependencia directa y eliminó los avisos del análisis de código.

Uno de los aprendizajes más útiles fue que no todos los hallazgos de una herramienta son problemas, pero todos merecen una explicación. Separar los chequeos críticos de los informativos, documentar los riesgos aceptados y aislar las dependencias incompatibles en

su propio entorno permitió conservar la estabilidad del proyecto sin ignorar ninguna alerta. Hacia adelante, el siguiente paso natural es ejecutar estas mismas verificaciones de forma automática en cada pull request mediante integración continua.

Referencias

Deepchecks. (s. f.). *Deepchecks documentation*. Recuperado el 1 de noviembre de 2026, de https://docs.deepchecks.com/stable/getting-started/welcome.html

Grossmann, J., Wiesbrock, H., & Motta, M. (2021). *Testing ML-based systems*. ETSI.

Huyen, C. (2022). *Designing machine learning systems: An iterative process for production-ready applications*. O'Reilly Media.

PyCQA. (2026). *Bandit* (Versión 1.9.4) [Software]. Python Package Index. https://pypi.org/project/bandit/

Python Packaging Authority. (2026). *pip-audit* (Versión 2.10.1) [Software]. Python Package Index. https://pypi.org/project/pip-audit/

pytest-dev. (s. f.). *pytest documentation*. Recuperado el 1 de noviembre de 2026, de https://docs.pytest.org/en/stable/

Wilson, B. (2022). *Machine learning engineering in action*. Manning Publications.